

JavaScript Arrays — Fundamentals & Creation

JavaScript-এ **Array** হলো এমন একটি data structure যেখানে একাধিক value একটি ordered collection হিসেবে রাখা যায়।

যেমন:

```
const fruits = ["Apple", "Banana", "Mango"];

console.log(fruits);
```

একটি array-এর প্রতিটি value-এর একটি নির্দিষ্ট **index** থাকে।

Index:	0	1	2
	↓	↓	↓
Value:	Apple	Banana	Mango

1. What is an Array?

একটি variable-এ একাধিক related value রাখার জন্য array ব্যবহার করা হয়।

Without array:

```
const fruit1 = "Apple";
const fruit2 = "Banana";
const fruit3 = "Mango";
```

Array দিয়ে:

```
const fruits = ["Apple", "Banana", "Mango"];
```

এটা code-কে অনেক cleaner করে।

2. Array Creation

সবচেয়ে common এবং recommended syntax হলো **array literal**:

```
const fruits = ["Apple", "Banana", "Mango"];
```

আরেকটি:

```
const numbers = [10, 20, 30, 40];
```

Boolean:

```
const values = [true, false, true];
```

3. Empty Array

Empty array:

```
const fruits = [];
```

পরে element add করা যায়:

```
const fruits = [];  
  
fruits[0] = "Apple";  
fruits[1] = "Banana";  
  
console.log(fruits);
```

Result:

```
["Apple", "Banana"]
```

4. Array-এ Different Data Types রাখা যায়

JavaScript array-এ একই ধরনের data রাখতেই হবে—এমন কোনো rule নেই।

```
const data = [  
  "Shariar",  
  31,  
  true,  
  null,  
  undefined  
];
```

এমনকি object এবং function-ও রাখা যায়:

```
const data = [  
  "Hello",  
  { name: "Shariar" },  
  [10, 20],  
  function () {  
    console.log("Hello");  
  }  
];
```

তবে practical application-এ সাধারণত related data একই structure-এ রাখা ভালো।

5. Array Indexing

Array-এর index ০ থেকে শুরু হয়।

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango"  
];
```

Index:

```
Apple    → 0  
Banana   → 1  
Mango    → 2
```

Access:

```
console.log(fruits[0]);  
    // Apple  
  
console.log(fruits[1]);  
    // Banana  
  
console.log(fruits[2]);  
    // Mango
```

6. Why Does Array Start at 0?

Programming languages-এ indexing সাধারণত zero-based।

তাই:

```
First element → index 0  
Second element → index 1  
Third element → index 2
```

এটা খুব important কারণ array নিয়ে কাজ করার সময় প্রায় সব loop/index calculation এই rule-এর উপর ভিত্তি করে।

7. Accessing the Last Element

ধরো:

```
const fruits = ["Apple", "Banana", "Mango"];
```

Last element:

```
console.log(fruits[fruits.length - 1]);
```

Result:

```
Mango
```

কারণ:

```
length = 3  
last index = 3 - 1 = 2
```

8. `length` Property

Array-এর element সংখ্যা জানতে `.length` ব্যবহার করা হয়।

```
const fruits = ["Apple", "Banana", "Mango"];  
  
console.log(fruits.length);  
// 3
```

আরেকটি:

```
const numbers = [10, 20, 30, 40, 50];  
  
console.log(numbers.length);  
// 5
```

9. `length` Method নয়

এটা মনে রাখা খুব important।

ভুল:

```
fruits.length();
```

সঠিক:

```
fruits.length;
```

কারণ `length` একটি **property**, function/method নয়।

10. Empty Array-এর Length

```
const fruits = [];  
  
console.log(fruits.length);  
// 0
```

11. Updating Array Elements

Array element-এর value পরিবর্তন করা যায়।

```
const fruits = ["Apple", "Banana", "Mango"];  
  
fruits[1] = "Orange";  
  
console.log(fruits);
```

Result:

```
["Apple", "Orange", "Mango"]
```

12. Adding an Element Using Index

```
const fruits = ["Apple", "Banana"];  
  
fruits[2] = "Mango";  
  
console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango"]
```

13. Index Skip করলে কী হয়?

এখানে একটু interesting behavior আছে:

```
const fruits = [];  
  
fruits[2] = "Mango";  
  
console.log(fruits);
```

Array দেখতে হবে roughly:

```
[empty, empty, "Mango"]
```

এবং:

```
console.log(fruits.length);  
// 3
```

অর্থাৎ index **2** পর্যন্ত যাওয়ার কারণে length **3** হয়েছে।

14. Sparse Array

যে array-এর কিছু index-এ actual element নেই, তাকে **sparse array** বলা হয়।

Example:

```
const numbers = [];  
  
numbers[0] = 10;  
numbers[5] = 60;  
  
console.log(numbers);
```

মাঝখানে কিছু empty slot থাকবে।

সাধারণ application code-এ অকারণে sparse array তৈরি করা avoid করা ভালো।

15. Out-of-Range Index

Array-এর existing range-এর বাইরে কোনো index access করলে:

```
const fruits = ["Apple", "Banana"];

console.log(fruits[10]);
```

Result:

```
undefined
```

এতে error হয় না।

16. Negative Index

Traditional bracket indexing-এ:

```
const fruits = ["Apple", "Banana", "Mango"];

console.log(fruits[-1]);
```

এটা last element দেবে না।

Result:

```
undefined
```

কারণ `-1` এখানে normal array index হিসেবে কাজ করে না।

Modern JavaScript-এ last element পাওয়ার জন্য `at()` ব্যবহার করা যায়:

```
console.log(fruits.at(-1));

// Mango
```

17. Nested Arrays

একটি array-এর ভিতরে আরেকটি array রাখা যায়।

```
const numbers = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];
```

এটাকে nested বা multidimensional array বলা যায়।

18. Nested Array Access

```
const numbers = [  
  [1, 2, 3],  
  [4, 5, 6]  
];
```

প্রথম row:

```
console.log(numbers[0]);
```

Result:

```
[1, 2, 3]
```

প্রথম row-এর দ্বিতীয় value:

```
console.log(numbers[0][1]);
```

Result:

```
2
```

Mental model:

```
numbers[0]      → first inner array  
numbers[0][1]   → first array-এর second value
```

19. `typeof` Array

এখানে একটি famous JavaScript behavior আছে:

```
const fruits = ["Apple", "Banana"];

console.log(typeof fruits);
```

Result:

```
object
```

অনেক beginner অবাক হয়।

কারণ JavaScript-এ array technically object-এর একটি special type।

20. তাহলে Array কিনা কীভাবে Check করব?

এখানেই `Array.isArray()` ব্যবহার করতে হয়।

```
const fruits = ["Apple", "Banana"];

console.log(Array.isArray(fruits));

// true
```

21. `Array.isArray()`

Syntax:

```
Array.isArray(value);
```

Examples:

```
Array.isArray([]);  
// true  
  
Array.isArray([1, 2, 3]);  
// true  
  
Array.isArray("hello");  
// false  
  
Array.isArray(100);  
// false  
  
Array.isArray({});  
// false
```

22. `typeof` VS `Array.isArray()`

Value	<code>typeof</code>	<code>Array.isArray()</code>
<code>[]</code>	<code>"object"</code>	<code>true</code>
<code>{}</code>	<code>"object"</code>	<code>false</code>
<code>"hello"</code>	<code>"string"</code>	<code>false</code>
<code>100</code>	<code>"number"</code>	<code>false</code>
<code>null</code>	<code>"object"</code>	<code>false</code>

Array detect করার জন্য:

```
Array.isArray(value)
```

ব্যবহার করাই correct।

23. Why `Array.isArray()` is Important

API থেকে data পাওয়ার সময় তুমি নিশ্চিত নাও হতে পারো data array কিনা।

```
const data = getDataFromAPI();

if (Array.isArray(data)) {
  console.log("We received an array");
}
```

এটা defensive programming-এর একটি useful pattern।

24. `Array.isArray()` with Function

```
function processData(data) {
  if (!Array.isArray(data)) {
    return;
  }

  console.log(data);
}
```

এতে function ভুল data structure পেলো safely handle করতে পারে।

25. `Array.from()`

`Array.from()` কোনো **iterable** বা **array-like object** থেকে নতুন array তৈরি করতে পারে।

Basic example:

```
const text = "HELLO";

const chars = Array.from(text);

console.log(chars);
```

Result:

```
["H", "E", "L", "L", "O"]
```

26. `Array.from()` with String

String iterable হওয়ায়:

```
const word = "JavaScript";

const letters = Array.from(word);

console.log(letters);
```

Result:

```
["J", "a", "v", "a", "s", "c", "r", "i", "p", "t"]
```

27. `Array.from()` with Set

`Set` থেকেও array তৈরি করা যায়।

```
const numbers = new Set([10, 20, 30]);

const array = Array.from(numbers);

console.log(array);
```

Result:

```
[10, 20, 30]
```

28. `Array.from()` with Map

```
const map = new Map([
  ["name", "Shariar"],
  ["age", 31]
]);

const entries = Array.from(map);

console.log(entries);
```

Result:

```
[
  ["name", "Shariar"],
  ["age", 31]
]
```

29. Array-Like Objects

সব object array নয়, কিন্তু কিছু object-এর:

- numeric indexes থাকে
- `length` property থাকে

এগুলোকে **array-like objects** বলা হয়।

Example:

```
const arrayLike = {
  0: "Apple",
  1: "Banana",
  2: "Mango",
  length: 3
};
```

এটা array নয়:

```
console.log(Array.isArray(arrayLike));
// false
```

কিন্তু `Array.from()` দিয়ে array বানানো যায়:

```
const fruits = Array.from(arrayLike);

console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango"]
```

30. DOM-এর সাথে `Array.from()`

Browser development-এ `NodeList` বা কিছু array-like collection থেকে real array বানাতে `Array.from()` useful।

Example:

```
const elements = document.querySelectorAll(".item");

const items = Array.from(elements);
```

এখন `items` একটি actual array।

31. `Array.from()` দ্বিতীয় Argument

`Array.from()` শুধু array তৈরি করে থামে না। প্রতিটি element-এর উপর mapping function চালানো যায়।

```
const numbers = Array.from(
  [1, 2, 3],
  (number) => number * 2
);

console.log(numbers);
```

Result:

```
[2, 4, 6]
```

এটা conceptually `map()` -এর মতো।

32. `Array.from()` দিয়ে Number Sequence

```
const numbers = Array.from(  
  { length: 5 },  
  (_, index) => index + 1  
);  
  
console.log(numbers);
```

Result:

```
[1, 2, 3, 4, 5]
```

এটা frontend utility এবং pagination-related code-এ useful হতে পারে।

33. `Array.of()`

`Array.of()` arguments নিয়ে একটি নতুন array তৈরি করে।

```
const numbers = Array.of(10, 20, 30);  
  
console.log(numbers);
```

Result:

```
[10, 20, 30]
```

34. `Array.of()` vs Array Literal

এই দুইটা:

```
Array.of(10, 20, 30);
```

এবং:

```
[10, 20, 30];
```

দুটোই array তৈরি করে।

Daily code-এ array literal:

```
[10, 20, 30]
```

বেশি common।

35. Why `Array.of()` Exists

একটি interesting difference হলো `Array()` ।

```
Array(3);
```

এটা 3 elements-এর একটি sparse array তৈরি করে:

```
[empty, empty, empty]
```

কিন্তু:

```
Array.of(3);
```

এটা:

```
[3]
```

তৈরি করে।

36. `Array()` vs `Array.of()`

```
Array(3);  
// [empty × 3]  
  
Array.of(3);  
// [3]
```

আর:

```
Array(1, 2, 3);  
// [1, 2, 3]  
  
Array.of(1, 2, 3);  
// [1, 2, 3]
```

এই কারণেই `Array.of()` constructor-এর single-number behavior avoid করার একটি predictable way।

37. Array Reference Behavior

এটা খুব important JavaScript concept।

```
const fruits = ["Apple", "Banana"];  
  
const anotherFruits = fruits;  
  
anotherFruits[0] = "Mango";  
  
console.log(fruits);
```

Result:

```
["Mango", "Banana"]
```

কেন?

কারণ `anotherFruits` নতুন array তৈরি করেনি।

দুটো variable একই array-কে reference করছে।

38. Reference Mental Model

এভাবে ভাবতে পারো:

```
fruits ———┐
              ↓
            ["Apple", "Banana"]
              ↑
          ┌───┴───┐
anotherFruits
```

তাই:

```
anotherFruits[0] = "Mango";
```

করলে original array-ও পরিবর্তিত হয়।

39. Array Copy — Basic Introduction

নতুন shallow copy তৈরি করতে spread ব্যবহার করা যায়:

```
const fruits = ["Apple", "Banana"];

const copy = [...fruits];

copy[0] = "Mango";

console.log(fruits);
// ["Apple", "Banana"]

console.log(copy);
// ["Mango", "Banana"]
```

এখানে:

```
copy !== fruits;
// true
```

Spread নিয়ে বিস্তারিত পরের Doc-এ আসবে।

40. Array Copy with `Array.from()`

`Array.from()` দিয়েও shallow copy করা যায়:

```
const fruits = ["Apple", "Banana"];

const copy = Array.from(fruits);

console.log(copy);
```

এখানে:

```
copy === fruits;
// false
```

অর্থাৎ নতুন array তৈরি হয়েছে।

41. Shallow Copy কী?

ধরো:

```
const users = [
  { name: "A" },
  { name: "B" }
];

const copy = [...users];
```

Outer array নতুন হয়েছে।

কিন্তু inner objects একই reference:

```
copy
  ↓
[new array]
  ↓
same object references
```

তাই:

```
copy[0].name = "C";

console.log(users[0].name);
// "C"
```

এটা **shallow copy**।

Deep copy একটি আলাদা concept, যা nested data নিয়ে কাজ করার সময় গুরুত্বপূর্ণ।

42. Arrays Are Objects

এই কারণে array-এর উপর object-এর মতো কিছু behavior দেখা যায়।

```
const fruits = ["Apple", "Banana"];

console.log(typeof fruits);
// "object"
```

কিন্তু array-এর special methods এবং indexing behavior আছে:

```
fruits.push("Mango");

console.log(fruits);
```

43. Array Properties

সবচেয়ে important built-in property:

```
array.length
```

Example:

```
const numbers = [10, 20, 30];

console.log(numbers.length);
// 3
```

Array-এর prototype-এ অনেক methods আছে, যেগুলো পরের docs-এ আসবে।

44. Array Index and Length Relationship

যদি array empty না হয় এবং normal contiguous array হয়:

```
last index = length - 1
```

Example:

```
const numbers = [10, 20, 30, 40];

console.log(numbers.length);
// 4

console.log(numbers[3]);
// 40
```

কারণ:

```
4 - 1 = 3
```

45. Changing `length`

JavaScript-এ `length` property writable।

```
const numbers = [10, 20, 30, 40, 50];

numbers.length = 3;

console.log(numbers);
```

Result:

```
[10, 20, 30]
```

শেষের elements বাদ পড়ে গেছে।

46. Increasing `length`

```
const numbers = [10, 20];

numbers.length = 5;

console.log(numbers);
```

Array-এর `length` হবে:

```
5
```

কিন্তু নতুন positions empty slots হবে।

তাই `length` manually change করার সময় সতর্ক থাকতে হবে।

47. Array Destructuring-এর Preview

Array থেকে values আলাদা variable-এ নেওয়া যায়:

```
const numbers = [10, 20, 30];

const [first, second, third] = numbers;

console.log(first);
// 10

console.log(second);
// 20

console.log(third);
// 30
```

এটা **array destructuring**।

এর বিস্তারিত পরের Doc-এ থাকবে।

48. Common Mistake — `typeof []`

ভুল assumption:

```
typeof [];  
// "array"
```

আসলে:

```
typeof [];  
// "object"
```

Correct:

```
Array.isArray([]);  
// true
```

49. Common Mistake — Last Index

ভুল:

```
const numbers = [10, 20, 30];  
  
numbers[numbers.length];
```

Result:

```
undefined
```

কারণ:

```
length = 3  
valid indexes = 0, 1, 2
```

সঠিক:

```
numbers[numbers.length - 1];  
// 30
```

50. Common Mistake — Array Copy

ভুল:

```
const original = [1, 2, 3];

const copy = original;

copy.push(4);

console.log(original);
// [1, 2, 3, 4]
```

কারণ একই reference।

Basic shallow copy:

```
const copy = [...original];
```

অথবা:

```
const copy = Array.from(original);
```

51. Common Mistake — **Array(3)**

```
const numbers = Array(3);

console.log(numbers);
```

এটা **[3]** নয়।

এটা length **3**-এর sparse array।

যদি **[3]** চাও:

```
const numbers = Array.of(3);
```

অথবা:

```
const numbers = [3];
```

52. `Array.from()` VS `Array.of()`

Feature	<code>Array.from()</code>	<code>Array.of()</code>
Main purpose	Existing iterable/array-like থেকে array	Arguments থেকে array
String convert	✓	✗
Set convert	✓	✗
Array-like convert	✓	✗
Multiple values	Can use source	✓
<code>Array.of(3)</code>	—	<code>[3]</code>
<code>Array(3)</code>	—	Different behavior

সহজভাবে:

`Array.from()`

→ "এটা থেকে একটা array বানাও"

`Array.of()`

→ "এই values দিয়ে একটা array বানাও"

53. Array Fundamentals Mental Model

Array

- |
 - ├ Ordered collection
 - |
 - ├ Zero-based indexing
 - | └ [0]
 - | └ [1]
 - | └ [2]
 - |
 - ├ length
 - |
 - ├ Can contain different values
 - |
 - ├ Can contain nested arrays
 - |
 - ├ typeof → "object"
 - |
 - ├ Array.isArray() → actual array check
 - |
 - ├ Array.from()
 - | └ iterable / array-like → array
 - |
 - └ Array.of()
 - | └ arguments → array

54. Quick Revision

```
const fruits = ["Apple", "Banana", "Mango"];
```

First element

```
fruits[0];
```

Last element

```
fruits[fruits.length - 1];
```

Length

```
fruits.length;
```

Check array

```
Array.isArray(fruits);
```

String → Array

```
Array.from("Hello");
```

Set → Array

```
Array.from(new Set([1, 2, 3]));
```

Create array from values

```
Array.of(10, 20, 30);
```

Shallow copy

```
const copy = [...fruits];
```

55. Practice — Predict the Output

Question 1

```
const fruits = ["Apple", "Banana", "Mango"];

console.log(fruits[0]);
```

Question 2

```
const fruits = ["Apple", "Banana", "Mango"];

console.log(fruits[fruits.length - 1]);
```

Question 3

```
const fruits = ["Apple", "Banana"];

console.log(fruits[5]);
```

Question 4

```
console.log(typeof []);
```

Question 5

```
console.log(Array.isArray([]));
```

Question 6

```
console.log(Array.isArray({}));
```

Question 7

```
console.log(Array.from("ABC"));
```

Question 8

```
console.log(Array.of(3));
```

Question 9

```
console.log(Array(3));
```

Question 10

```
const a = [1, 2, 3];  
const b = a;  
  
b[0] = 100;  
  
console.log(a);
```

56. Coding Practice

Challenge 1 — Get Last Element

Function লিখো:

```
function getLastElement(array) {  
  // return last element  
}  
  
console.log(  
  getLastElement(["A", "B", "C"])  
);  
  
// "C"
```

Challenge 2 — Check Array

```
function isArray(value) {  
    // return true if value is an array  
}
```

Expected:

```
isArray([1, 2, 3]);  
// true  
  
isArray("hello");  
// false  
  
isArray({});  
// false
```

Challenge 3 — String to Array

```
function stringToArray(value) {  
    // convert string to array  
}  
  
console.log(stringToArray("HELLO"));
```

Expected:

```
["H", "E", "L", "L", "O"]
```

Challenge 4 — Create Number Range

`Array.from()` ব্যবহার করে:

```
createRange(1, 10);
```

Expected:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

57. Interview Questions

Beginner

1. JavaScript array কী?
2. Array indexing কোথা থেকে শুরু হয়?
3. Array-এর last index কীভাবে বের করবে?
4. `.length` কী?
5. `.length` কি method?
6. Array-এর ভিতরে different data types রাখা যায়?
7. `typeof []` কী return করে?
8. Array check করার জন্য `Array.isArray()` কেন ব্যবহার করা হয়?

Intermediate

9. `Array.isArray()` এবং `typeof` -এর পার্থক্য কী?
10. `Array.from()` কী?
11. `Array.from()` এবং `Array.of()` -এর পার্থক্য কী?
12. `Array(3)` এবং `Array.of(3)` -এর পার্থক্য কী?
13. Array-like object কী?
14. Sparse array কী?
15. Array reference behavior কী?
16. Shallow copy কী?
17. `[...array]` এবং `Array.from(array)` কীভাবে array copy করে?
18. Nested array কী?

58. Golden Rules

1. Array index সবসময় `0` থেকে শুরু হয়।

2. Last index = `array.length - 1`

3. `.length` একটি *property, method* নয়।

4. `typeof []` → `"object"`

5. Array detect করতে `Array.isArray()` ব্যবহার করো।

6. `Array.from()` → iterable বা array-like data থেকে নতুন array তৈরি করে।

7. `Array.of()` → supplied arguments দিয়ে নতুন array তৈরি করে।

8. `Array(3)` এবং `Array.of(3)` এক জিনিস নয়।

9. `const b = a` করলে array copy হয় না; একই reference থাকে।

10. `[...array]` বা `Array.from(array)` basic shallow copy তৈরি করতে পারে।

11. Nested objects/arrays থাকলে shallow copy-এর limitation মনে রাখতে হবে।

12. Normal array access-এর বাইরে index দিলে `undefined` পাওয়া যায়।